◆ PROJECT DOCUMENTATION

Linux AI Assistant

A locally-run, LLM-powered terminal assistant that translates natural-language instructions into validated Linux commands and streams their output directly to an in-browser terminal — with Ollama as the primary inference engine.

STACK	LLM	TERMINAL	VERSION
Node.js · React 19 · Vite	Ollama (local) · Gemini (fallback)	xterm.js + node-pty	1.0.0

1 Overview

Linux AI Assistant is a self-hosted web application that bridges natural language and the Linux shell. You type a task in plain English — "update system", "show open ports", "find files larger than 100MB" — and the assistant generates the correct shell command, validates it through a 3-stage safety pipeline, and streams real PTY output to a live in-browser terminal.

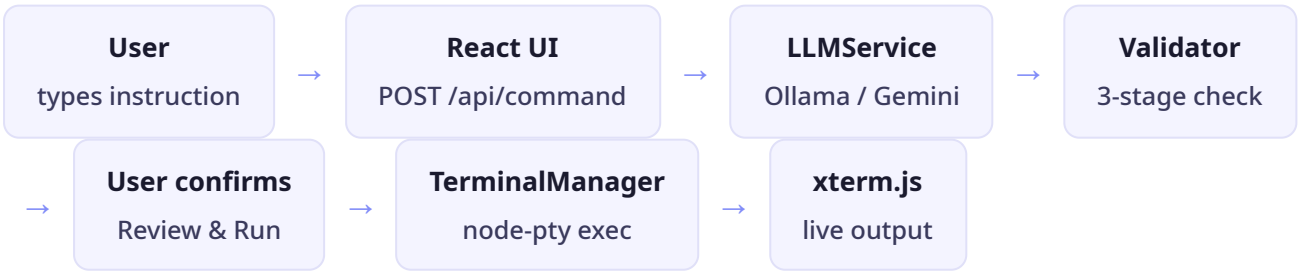
All inference runs locally via **Ollama**. The Gemini API is only contacted if Ollama is completely unreachable (e.g. the service is down). No command data is sent to external servers during normal operation.

- **Local-first inference**
  Ollama runs models on your own hardware. Commands are generated without any cloud dependency.
- **3-stage validation**
  Every command passes bash syntax check → regex blacklist → LLM semantic safety before execution.
- **Live PTY terminal**
  xterm.js connects to a real node-pty shell. Progress bars, colours, and interactive prompts all work.
- **Gemini fallback**
  If Ollama is unreachable, generation falls back to Gemini automatically — no manual switching needed.

2 Architecture

The project is a monorepo with two workspaces: a Node.js/Express backend and a React frontend.

Request flow



Backend services

Service	Responsibility
`LLMService`	Provider orchestration — tries Ollama, falls back to Gemini on network error

Service	Responsibility
`CommandProcessor`	Orchestrates generation → validation → execution lifecycle
`CommandValidator`	Runs syntax check, regex blacklist (400+ patterns), LLM semantic check
`TerminalManager`	Manages node-pty sessions, PTY resize, buffered output emission
`SessionManager`	In-memory session store with TTL-based cleanup
`SystemProfile`	Detects OS, package manager, shell — used to tune LLM prompts
`RAG / knowledge.json`	Fuzzy-matched known-task database to speed up common commands

Frontend components

Component	Purpose
`Index.jsx`	Root page — navbar, chat panel, sidebar, terminal section
`CommandBlock.jsx`	Renders generated command with validation chips, explanation, actions
`TerminalPanel.jsx`	xterm.js terminal with FitAddon, PTY resize sync, macOS-style title bar
`SettingsDrawer.jsx`	Slide-in drawer — model selector, provider info, session management
`ConfirmModal.jsx`	Pre-execution review modal for high-risk / sudo commands
`useWebSocket.jsx`	Socket.IO hook — terminal I/O, resize, command status events

3 Safety & Validation

No command reaches the shell without passing all three validation stages:

1. **Bash syntax check** — `bash -n` parses the command without executing it. Syntax errors block execution immediately.
2. **Regex blacklist** — 400+ patterns block known destructive patterns: `rm -rf /`, fork bombs, raw disk writes (`dd if=`), kernel module tampering, and more.
3. **LLM semantic check** — The local Ollama model reviews the command in context and assigns a risk level (low / medium / high). High-risk commands require explicit user confirmation.

> **Confirmation gate:** Commands flagged as requiring confirmation open a Review & Run modal that shows the full command, risk level, and validation reasons before anything is sent to the shell.

Security hardening

- CORS enforced with strict origin allowlist — `null` origins rejected
- Helmet.js with explicit CSP, HSTS (1 year), `X-Frame-Options: DENY`
- PTY environment uses an explicit whitelist — no `process.env` spread (prevents secret leakage to child processes)
- UUID-validated `messageId` and `sessionId` on all API routes
- Socket.IO per-socket rate limiting (100 events / 10 seconds)

4 LLM Provider Strategy

Ollama — Primary

All command generation, semantic validation, explanation, and failure diagnosis runs through Ollama locally. No data leaves the machine.

Gemini — Network fallback only

Gemini is contacted only if Ollama throws a network error (service down). It is never used as a quality fallback or on syntax failures.

The `LLMService.tryProviders()` method handles the fallback naturally: it catches network-level errors from Ollama and tries the next provider. Application-level failures (bad output, parse errors) are not retried with Gemini.

5 Installation & Running

Prerequisites

- Node.js 18+ and npm
- Ollama installed and running (`ollama serve`)
- At least one Ollama model pulled: `ollama pull mistral`
- Optional: Gemini API key in `backend/.env` for the fallback

Quick start

```
# Clone and install
git clone <repo-url>
cd LLM-Powered-Autonomous-Agent
npm install

# Configure (copy example and add your Gemini key if needed)
cp backend/.env.example backend/.env

# Start both backend and frontend
./start.sh
```

```
# Open in browser
http://localhost:5173
```

Environment variables

Variable	Default	Description
`OLLAMA_URL`	`http://localhost:11434`	Ollama API base URL
`OLLAMA_MODEL`	`mistral`	Default model for generation
`GEMINI_API_KEY`	—	Gemini API key (fallback only)
`PORT`	`3000`	Backend server port
`ALLOWED_ORIGINS`	`http://localhost:5173`	CORS allowlist

6 UI Overview

The interface is a minimal developer-tool aesthetic — flat dark theme with Geist fonts, HSL design tokens, and purposeful whitespace. No gradients, no animations for their own sake.

- **Navbar** — Logo mark, connection status badge, active model chip, refresh and settings buttons
- **Chat panel** — Conversation history with user messages (right-aligned) and assistant command cards (left)
- **Command card** — Command in a dark code block, validation dot chips, risk-level top border accent, explanation, alternatives, Copy / Explain / Run actions
- **Sidebar** — Environment status (Ollama, Gemini, system info) and execution history
- **Terminal** — Full xterm.js terminal with PTY resize sync, catppuccin colour scheme, macOS-style title bar
- **Settings drawer** — Slides in from right; model selector, provider policy display, session clear

7 Project Structure

```
LLM-Powered-Autonomous-Agent/
├── backend/
│   ├── config/          # env, logger, blacklist.json
│   ├── routes/          # Express API router
│   ├── services/        # LLMService, CommandProcessor, TerminalManager …
│   │   └── llm/providers/  # OllamaProvider, GeminiProvider
│   ├── rag/             # knowledge.json + search/update scripts
│   ├── tests/           # Unit tests (Jest)
```

```
│   ├── server.js
│   └── .env.example
├── frontend/
│   ├── src/
│   │   ├── components/  # CommandBlock, TerminalPanel, ChatMessage …
│   │   ├── hooks/       # useWebSocket
│   │   ├── lib/         # api, session, runtime helpers
│   │   └── pages/       # Index.jsx (root page)
│   ├── index.css        # Design token system
│   └── vite.config.js
├── install.sh
├── start.sh
└── package.json
```